

Reviewer Pack — Operator Norm Separation for Read-Twice Branching Programs

Entry bac847b867fa · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-08 13:14:16 UTC

Operator Norm Separation for Read-Twice Branching Programs

Entry ID: bac847b867fa **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-08 13:14:16 UTC

1. Conjecture

Field A (mathematical branch): OPERATOR_THEORY **Field B** (complexity object): BRANCHING_PROGRAMS

Statement:

For any read-twice branching program P computing a function f , the spectral norm of the associated transition matrix M_P is $O(\log \text{size}(P))$, whereas for the trivial read-twice BP computing the inner product mod 2 (IP_2), the spectral norm is $\Omega(n)$.

Rationale (proposer's reasoning):

The spectral norm of the transition matrix captures the 'spread' of the BP's transitions. Read-twice BPs, being more constrained, have matrices with smaller spectral norms, while the IP_2 function, which requires exponential size, has a much larger norm. This separation could lead to lower bounds on the size of read-twice BPs for certain functions.

Taxonomy category: PROOF_COMPLEXITY_TSEITIN (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9a3a44f3356b7336

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	UNCERTAIN	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=1.7s

5.1 Generated Python source

```
import random
import math

def gaussian_elimination(A):
    n = len(A)
    for i in range(n):
        max_row = i + max(range(i, n), key=lambda j: abs(A[j][i]))
        A[i], A[max_row] = A[max_row], A[i]
        for j in range(i+1, n):
            factor = A[j][i] / A[i][i]
            for k in range(n):
                A[j][k] -= factor * A[i][k]
    return A

def spectral_norm(M):
    n = len(M)
    v = [random.random() for _ in range(n)]
    v /= math.sqrt(sum(x**2 for x in v))
    for _ in range(100): # Power iteration
        v = [sum(M[i][j] * v[j] for j in range(n)) for i in range(n)]
        v /= math.sqrt(sum(x**2 for x in v))
    return max(abs(v))

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = 5 + (seed % 40) // 8
    size_P = 2 ** n

    # Construct transition matrix M_P for a read-twice BP
    M_P = [[0] * size_P for _ in range(size_P)]
    for i in range(n):
        for j in range(1 << i):
            for k in range(1 << (n - i)):
                M_P[j][j ^ k] += 1
                M_P[j ^ k][j] += 1
    M_P = gaussian_elimination(M_P)

    norm_M_P = spectral_norm(M_P)
```

```

# Construct transition matrix for IP_2 function
M_IP2 = [[0] * (1 << n) for _ in range(1 << n)]
for i in range(1 << n):
    for j in range(1 << n):
        if bin(i ^ j).count('1') % 2 == 0:
            M_IP2[i][j] = 1
M_IP2 = gaussian_elimination(M_IP2)

norm_M_IP2 = spectral_norm(M_IP2)

return {
    "metric_name": "spectral_norm",
    "metric_value": norm_M_IP2 / math.log(size_P),
    "instances_tested": n,
    "conjecture_holds": norm_M_IP2 > 10 * math.log(size_P),
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [random.randint(1, 1000000) for _ in range(30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    std_value = math.sqrt(sum((r["metric_value"] - mean_value) ** 2 for r in results) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample=\"norm_M_IP2 <= 10 * log(size_P)\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b091259.py", line 78, in <module>

```

    result = run_trial(seed)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b091259.py", line 50, in run_trial

```

    M_P = gaussian_elimination(M_P)
    ~~~~~

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_5b091259.py", line 24, in gaussian_elimination

```
factor = A[j][i] / A[i][i]
          ~~~~~^~~~~~
```

ZeroDivisionError: float division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed with ZeroDivisionError, preventing result collection. No data to assess conjecture validity. | next: Debug gaussian_elimination() to handle singular matrices, then re-run with robust error checking

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	77604
2	propose	ollama_remote	qwen3:8b	0	0	48613
3	propose	ollama_remote	qwen3:8b	0	0	73064
4	preregistration	ollama_remote	qwen3:8b	0	0	28474
5	novelty	ollama_remote	qwen3:8b	0	0	20785
6	novelty	ollama_remote	qwen3:8b	0	0	13625
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16979
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31686
9	judge	ollama_remote	qwen3:8b	0	0	118363

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 429192 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/bac847b867fa.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/bac847b867fa.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/bac847b867fa.tar.gz (if generated)